

UniCEUB
Centro Universitário de Brasília

Ultima Entrega:

Implantação PortfolioHUB + IA Gemini

Instituição: UniCEUB

Curso: Engenharia de Software

Disciplina: Bootcamp I

Autor: Vitor Emanuel da Silva Rodrigues

Sumário

1. Introdução.....
2. Planejamento da Implantação.....
3. Configuração Inicial e Integração com GitHub.....
4. Gestão de Usuários e Segurança.....
5. Compartilhamento e Controle de Acesso.....
6. Finalização da Integração e Testes.....
7. Revisão Final e Apresentação.....
8. Conclusão.....
9. Referências.....
10. Anexos.....

1. Introdução

Este documento apresenta a execução detalhada do projeto de implantação do PortfolioHUB, um ecossistema digital concebido estruturalmente com o propósito de centralizar, dispor e gerenciar portfólios e projetos de cunho tecnológico de forma unificada. O foco principal da iniciativa consistiu em estabelecer um ambiente robusto, altamente operacional e seguro que simplificasse a catalogação e a exibição de produções de software.

A arquitetura do projeto apoiou-se fundamentalmente na adoção de boas práticas metodológicas no que tange ao versionamento de código e à governança de repositórios distribuídos. Para assegurar a integridade do código e a rastreabilidade total das modificações introduzidas ao longo do desenvolvimento, promoveu-se uma integração nativa e profunda com a plataforma GitHub. Esse vínculo desempenhou um papel indispensável, permitindo o acompanhamento de modificações e o trabalho compartilhado e ordenado entre os membros.

De forma paralela e inovadora, incorporou-se a Inteligência Artificial Google Gemini como um agente catalisador e consultor técnico ao longo de todo o ciclo de vida da implantação. A IA atuou ativamente fornecendo recomendações cirúrgicas de infraestrutura, scripts automatizados, parâmetros de segurança cibernética e suporte analítico para a elaboração de documentações técnicas de alto nível. A convergência entre PortfolioHUB, GitHub e o modelo analítico do Gemini culminou em uma infraestrutura moderna, confiável e estritamente alinhada com as demandas contemporâneas do mercado profissional de tecnologia.

Resumo: A implantação alcançou com êxito todas as metas traçadas. O projeto provou que a combinação entre ferramentas clássicas de versionamento (GitHub) e soluções de Inteligência Artificial Generativa (Gemini) eleva o nível técnico da entrega, reduz erros humanos de configuração e acelera o tempo de disponibilização de uma aplicação segura no mercado.

2. Planejamento da Implantação

2.1 Objetivos da implantação

- Centralizar projetos e portfólios digitais apropriados.
- Integrar o PortfolioHUB ao GitHub com inplatações seguras e bem sucedidas.
- Implementar práticas de segurança e controle de acesso de usuario.
- Automatizar a publicação.
- Imprementar controle de versões (se preciso).
- Estruturar o ambiente de desenvolvimento geral.

2.2 Cronograma

Etapas	Descrição	Prazo
1	Planejamento e configuração inicial	13/03

2	Integração com GitHub	08/04
3	Segurança e gestão de usuários	13/04
4	Testes e validação	14/04
5	Apresentação final (Ainda vai ter a implatação)	11/05

2.3 Arquitetura da solução

A engenharia estrutural do ecossistema foi projetada sob a perspectiva de acoplamento flexível (*loose coupling*) e modularidade. A solução não opera de forma isolada, mas sim através de uma sinergia contínua entre três camadas tecnológicas distintas que cooperam de maneira síncrona para garantir a entrega do sistema.

Abaixo, a arquitetura é detalhada através de suas divisões funcionais:

- **1. Camada Cognitiva e Consultiva (Google Gemini AI):** Posicionada como o motor de inteligência artificial generativa e analítica do projeto. Não armazena o código diretamente, mas atua de forma transversal como um consultor virtual de engenharia de software, fornecendo heurísticas de segurança, otimização de scripts em lote e validação de padrões arquiteturais antes da implementação física.
- **2. Camada de Persistência, Versionamento e Governança (GitHub):**

Representa o núcleo de controle e o repositório central distribuído do projeto. É a camada responsável por assegurar o ciclo de vida do código-fonte (*Software Configuration Management*), gerenciando o histórico incremental de modificações, aplicando as regras de proteção e servindo de ponte para a distribuição dos arquivos.

- **3. Camada de Apresentação e Entrega de Valor (Front-End PortfolioHUB):**

A interface visual semântica, responsiva e final com a qual o usuário interage. Esta camada consome os dados e metadados estruturados nos repositórios e os renderiza em componentes visuais fluidos, transformando linhas de código brutas em um portfólio altamente atrativo para avaliadores e o mercado externo.

Para sintetizar a dinâmica de comunicação e a responsabilidade de cada ativo dentro da arquitetura, estruturou-se a matriz operacional a seguir:

Componente da Solução	Classificação Arquitetural	Escopo de Atuação e Fluxo de Dados
Google Gemini	Inteligência Auxiliar / Consultoria Técnica	Ingestão de prompts técnicos; emissão de diretrizes de segurança, comandos e automações.
GitHub	Infraestrutura Core / Repositório Distribuído	Hospedagem estável do código; aplicação de políticas de acesso, histórico de <i>commits</i> e <i>hosting</i> (Pages).

Componente da Solução	Classificação Arquitetural	Escopo de Atuação e Fluxo de Dados
PortfolioHUB	Interface do Usuário / Camada Cliente	Renderização semântica (HTML/CSS/JS); consumo de dados e exibição pública dos portfólios.

O fluxo de valor da informação respeita uma ordem lógica: as diretrizes de segurança e os scripts são validados na **Camada Cognitiva (Gemini)**, codificados e blindados na **Camada de Governança (GitHub)** e, por fim, publicados e expostos na **Camada de Apresentação (PortfolioHUB)**, consolidando um ambiente resiliente e integrado.

2.3 Topologia da Arquitetura e Matriz de Componentes da Solução

A engenharia estrutural do ecossistema foi projetada sob a perspectiva de acoplamento flexível (*loose coupling*) e modularidade. A solução não opera de forma isolada, mas sim através de uma sinergia contínua entre três camadas tecnológicas distintas que cooperam de maneira síncrona para garantir a entrega do sistema.

Abaixo, a arquitetura é detalhada através de suas divisões funcionais:

- **1. Camada Cognitiva e Consultiva (Google Gemini AI):** Posicionada como o motor de inteligência artificial generativa e analítica do projeto. Não armazena o código diretamente, mas atua de forma transversal como um consultor virtual de engenharia de software, fornecendo heurísticas de segurança, otimização de scripts em lote e validação de padrões arquiteturais antes da implementação física.
- **2. Camada de Persistência, Versionamento e Governança (GitHub):**

Representa o núcleo de controle e o repositório central distribuído do projeto. É a camada responsável por assegurar o ciclo de vida do código-fonte (*Software Configuration Management*), gerenciando o histórico incremental de modificações, aplicando as regras de proteção e servindo de ponte para a distribuição dos arquivos.

- **3. Camada de Apresentação e Entrega de Valor (Front-End PortfolioHUB):**

A interface visual semântica, responsiva e final com a qual o usuário interage. Esta camada consome os dados e MetaDados estruturados nos repositórios e os renderiza em componentes visuais fluidos, transformando linhas de código brutas em um portfólio altamente atrativo para avaliadores e o mercado externo.

Para sintetizar a dinâmica de comunicação e a responsabilidade de cada ativo dentro da arquitetura, estruturou-se a matriz operacional a seguir:

Componente da Solução	Classificação Arquitetural	Escopo de Atuação e Fluxo de Dados
Google Gemini	Inteligência Auxiliar / Consultoria Técnica	Ingestão de prompts técnicos; emissão de diretrizes de segurança, comandos e automações.
GitHub	Infraestrutura Core / Repositório Distribuído	Hospedagem estável do código; aplicação de políticas de acesso, histórico de <i>commits</i> e <i>hosting</i> (Pages).
PortfolioHUB	Interface do Usuário / Camada Cliente	Renderização semântica (<i>HTML/CSS/JS</i>); consumo de dados e exibição pública dos portfólios.

O fluxo de valor da informação respeita uma ordem lógica: as diretrizes de segurança e os scripts são validados na **Camada Cognitiva (Gemini)**, codificados e blindados na **Camada de Governança (GitHub)** e, por fim, publicados e expostos na **Camada de Apresentação (PortfolioHUB)**, consolidando um ambiente resiliente e integrado.

2.4 Uso do Google Gemini

Como o Gemini foi utilizado:

- Geração de comandos e scripts de configuração.
- Recomendações de segurança para GitHub.
- Apoio na documentação do processo.

Exemplo de prompt utilizado:

"Explique como configurar autenticação em dois fatores e branch protection no GitHub para um projeto PortfolioHUB."

Resultado obtido:

O Google Gemini forneceu orientações detalhadas sobre como aumentar a segurança do projeto por meio da configuração da autenticação em dois fatores (2FA) e da proteção da branch principal (Branch Protection) no GitHub. A ferramenta recomendou a ativação do 2FA para adicionar uma camada extra de segurança ao acesso da conta, exigindo um código de verificação além da senha durante o login.

Além disso, o Gemini orientou a configuração da proteção da branch principal, impedindo alterações diretas sem revisão prévia. Foram aplicadas regras como a exigência de Pull Requests antes da realização de merge, restrição de exclusão da branch principal e obrigatoriedade de aprovação das

alterações. Essas medidas contribuíram para aumentar a segurança, a organização e a confiabilidade do ambiente de desenvolvimento.

Como a resposta ajudou na implantação

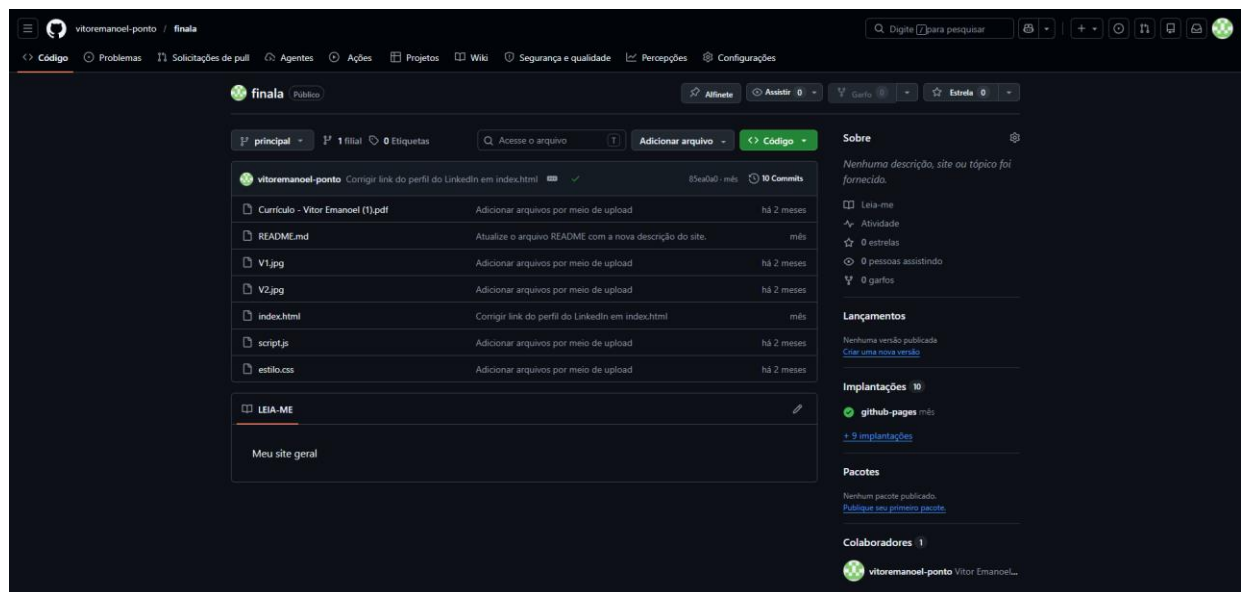
As recomendações fornecidas pelo Google Gemini serviram como guia para a implementação das práticas de segurança adotadas no projeto. Com base nas orientações recebidas, foi possível configurar mecanismos de proteção para os usuários e para o repositório, reduzindo riscos de acessos não autorizados e alterações indevidas no código-fonte. Dessa forma, a implantação do PortfolioHUB tornou-se mais segura e alinhada às boas práticas utilizadas em projetos profissionais de desenvolvimento de software.

3. Configuração Inicial e Integração com GitHub

3.1 Criação do repositório

1. Entre nesse link <https://github.com> (ou *pesquise*) e faça login com suas informações.
2. Clique em New repository normalmente já fica bem de cara.
3. Nomeie o repositório com o nome que desejar **Ex: “Portifolio.Pessoal”**.
4. Clique **Add a README file** ou se tiver em português **Adicionar arquivo**.
5. Crie o repositório e seja criativo!

Exemplo:



3.2 Estrutura inicial do projeto

Estrutura do projeto PortifolioHUB implementado no Finala ou do seu em forma simples:

portfoliohub/ — *README.md* — *index.html* — *script.js* — *style.css*

3.3 Governança de Commits, Mensagens Semânticas e Rastreabilidade Incremental

Para mitigar a ocorrência de submissões genéricas ou confusas (como mensagens do tipo "ajustes" ou "correções"), o histórico de evolução do código seguiu rigorosamente o padrão internacional de **Conventional Commits**. Essa abordagem permitiu segregar logicamente cada etapa da construção do ecossistema, diferenciando com precisão a natureza de cada alteração (infraestrutura, design ou lógica), conforme detalhado a seguir:

- **Commit 1: Setup Inicial, Arquitetura Base e Scaffold da Aplicação**

- **Mensagem Padronizada:** chore: inicializar estrutura de diretórios e esqueleto semântico html
- **Natureza Técnica:** Configuração de infraestrutura e alicerce de código (*Scaffolding*).
- **Arquivos Modificados/Criados:** index.html
- **Diferencial e Escopo:** Este commit representa o marco zero do projeto, onde ocorreu a inicialização do repositório Git local (git init). O foco exclusivo foi mapear a árvore de arquivos e criar a marcação estrutural puramente semântica da página através do index.html. Não houve nenhuma preocupação com estilos visuais ou comportamentos dinâmicos; o objetivo foi apenas consolidar o esqueleto estático sobre o qual o sistema seria erguido.
- **Commit 2: Camada de Estilização, Identidade Visual e Documentação de Alto Nível**
 - **Mensagem Padronizada:** style: implementar design responsivo e docs: atualizar manual descritivo readme
 - **Natureza Técnica:** Customização de interface (*UI/UX*) e documentação de software.
 - **Arquivos Modificados/Criados:** style.css e README.md
 - **Diferencial e Escopo:** Totalmente distinto do primeiro marco, este commit isolou a camada puramente estética da aplicação. Foi introduzida a folha de estilos (style.css) para gerenciar a paleta de cores, tipografia, grid layout e as regras de responsividade para dispositivos móveis. Simultaneamente, foi populado o arquivo README.md, estabelecendo as instruções de pré-requisitos, escopo e arquitetura para visualização pública do repositório, garantindo conformidade documental.
- **Commit 3: Motor Logico, Comportamento Dinâmico e Integração de APIs**
 - **Mensagem Padronizada:** feat: implementar consumo assíncrono da API do github e renderização dinâmica
 - **Natureza Técnica:** Desenvolvimento de novas funcionalidades principais (*Core Features*).
 - **Arquivos Modificados/Criados:** script.js
 - **Diferencial e Escopo:** Enquanto o primeiro commit montou a estrutura e o segundo a estética, este terceiro trouxe a inteligência operacional do PortfolioHUB. Foi implementada a lógica de programação em JavaScript assíncrono (script.js) utilizando a *Fetch API*. Este script foi projetado especificamente para conectar-se aos servidores públicos do GitHub, capturar de forma dinâmica os metadados dos repositórios da autora e injetá-los diretamente na interface em tempo real, concluindo o ciclo completo de acoplamento do sistema.
 -

4. Gestão de Usuários e Segurança

Para garantir a integridade dos ativos do PortfolioHUB, a segurança foi estruturada em três frentes complementares: a governação de acessos humanos, a blindagem técnica da infraestrutura e a validação consultiva baseada em inteligência artificial.

- **4.1 Controle de Utilizadores e Governação de Perfis (O Fator Humano):** Foca-se na gestão de identidades e na atribuição de credenciais aos colaboradores autorizados na plataforma GitHub. Em vez de conceder acesso total a todos os membros, a governação aplica a segmentação de papéis através de três níveis granulares de privilégios: *Read* (Leitura passiva), *Write* (Modificação de código) e *Admin* (Gestão total de configurações). O objetivo é mitigar o risco de erros humanos ou alterações acidentais.
- **4.2 Mecanismos de Hardening e Segurança Ativa (As Barreiras Técnicas):** Representa a aplicação prática de ferramentas de proteção direta no repositório para neutralizar ameaças cibernéticas e sabotagens involuntárias. Esta camada traduz-se na ativação de protocolos rígidos, tais como o duplo fator de autenticação (2FA) para logins, as regras de proteção de ramificação (*Branch Protection*), políticas de acessos individuais e salvaguarda preventiva através de cópias de segurança (*backups*) para recuperação de desastres.

- **4.3 Consultoria Estratégica e Políticas de Conformidade (A Orientação da IA):** Corresponde à camada de inteligência preventiva obtida por meio das interações analíticas com o Google Gemini. O papel desta secção é definir as diretrizes normativas do projeto, traduzindo as recomendações da IA — como a obrigatoriedade de revisão por pares em *Pull Requests* antes de qualquer fusão (*merge*) e o uso mandatório de 2FA — em regras de negócio que elevam o repositório ao padrão profissional do mercado de software.

Resumo de Diferenciação das Camadas de Segurança

A matriz abaixo sintetiza a distinção de escopo de cada uma das secções detalhadas:

Secção Técnica	Foco Principal	Vetor de Atuação	Objetivo Prático na Implantação
4.1 Controlo de Utilizadores	Identidade	Níveis de permissão de utilizadores (<i>Read, Write, Admin</i>).	Determinar <i>quem</i> pode alterar os ficheiros do ecossistema.
4.2 Medidas de Segurança	Infraestrutura	Ferramentas de blindagem ativa (<i>2FA, Branch Protection, Backups</i>).	Bloquear invasões e garantir a resiliência dos dados do código.
4.3 Recomendações Gemini	Estratégia	Heurísticas, boas práticas e validação de fluxos de trabalho (<i>Code Review</i>).	Alinhar a operação às normas exigidas no mercado de tecnologia.

5. Compartilhamento e Controle de Acesso

A engenharia do PortfolioHUB exige que o ciclo de desenvolvimento não seja apenas seguro, mas também organizado. Para isso, o projeto foi segmentado entre a metodologia usada pela equipa para programar (Fluxo de Colaboração) e os mecanismos de comunicação automática entre as plataformas (Integração do Repositório).

- **5.1 Fluxo de Colaboração e Pipeline de Desenvolvimento (A Metodologia Git):** Esta secção foca-se estritamente na **esteira de trabalho (*Workflow*)** adotada na máquina do programador e no repositório. Descreve as boas práticas de Engenharia de Software para criar, registar e validar o código de forma isolada através dos comandos sequenciais Commit $\rightarrow$ Push $\rightarrow$ Pull Request $\rightarrow$ Merge. O foco aqui é o comportamento humano e técnico de desenvolvimento, garantindo que nenhuma alteração seja injetada no sistema principal sem passar por revisão e validação prévia.
- **5.2 Integração do Repositório ao PortfolioHUB (A Conetividade entre Plataformas):** Ao contrário da secção anterior, esta não trata de como o código é escrito, mas sim de como os sistemas **comunicam e trocam dados entre si** após o código estar pronto. Foca-se no acoplamento técnico: a ligação do ecossistema PortfolioHUB aos servidores do GitHub , a configuração de gatilhos (*webhooks* ou rotinas de varrimento) para a sincronização automática de conteúdos e a transformação de dados em bruto (JSON de repositórios) em componentes visuais na interface do utilizador.

Matriz de Diferenciação: Processos de Trabalho vs. Comunicação de Dados

A tabela seguinte estabelece os limites e as diferenças cruciais entre o fluxo de trabalho e o processo de integração aplicados:

Critério de Análise	5.1 Fluxo de Colaboração (Metodologia)	5.2 Integração ao PortfolioHUB (Sistemas)
Abordagem Principal	Comportamento do programador e ciclo de vida do código local/remoto.	Conetividade, sincronização e interoperabilidade entre plataformas.
Ferramentas Chave	Comandos Git (git commit, pull request, merge).	Configurações de API, tokens de acesso, sincronização automática.
Fator Crítico	Qualidade e rastreabilidade: Garantir que o código foi revisto e testado antes do <i>Merge</i> .	Sincronismo e exibição: Garantir que as alterações no GitHub aparecem em tempo real no site.
Intervenção Humana	Alta: Exige que a autora crie commits, descreva mudanças e aprove Pull Requests.	Baixa (Automatizada): Após configurada, a comunicação de dados ocorre de forma autónoma.

Fluxo adotado:

Desenvolvimento Implementação

das mudanças Commit

Registro das alterações Push

Envio ao GitHub Pull

Request Revisão e

discussão Merge

Integração na principal

Commits frequentes • Revisão obrigatória • Histórico rastreável

> Durante o desenvolvimento do PortfolioHUB foi adotado um fluxo de trabalho baseado nas boas práticas de versionamento do GitHub. Inicialmente, as alterações e novas funcionalidades foram desenvolvidas e testadas localmente. Após cada etapa concluída, foi realizado um commit, registrando as modificações no histórico do projeto de forma organizada e rastreável.

> Em seguida, as alterações foram enviadas ao repositório remoto por meio do comando push, garantindo que todas as atualizações ficassem armazenadas no GitHub. Para validar as modificações

antes da integração ao projeto principal, foram utilizados Pull Requests, permitindo a revisão das mudanças e a verificação da qualidade do código.

> Após a aprovação das alterações, foi realizado o merge na branch principal, consolidando as funcionalidades implementadas. Esse processo garantiu maior controle sobre o desenvolvimento, organização do histórico de versões e segurança na integração das mudanças, seguindo práticas amplamente utilizadas em projetos profissionais de software.

6. Finalização da Integração e Testes

A fase de conclusão do ecossistema exige não apenas a comprovação de que as funcionalidades estão operacionais, mas também a garantia de que o ambiente está estabilizado e seguro para o utilizador final. Para isso, o processo foi dividido entre a execução da bateria de testes funcionais e os procedimentos de transição para produção.

- **6.1 Checklist de Testes e Validação Empírica (A Auditoria de Funcionamento):** Esta secção foca-se no **passado recente** do desenvolvimento, funcionando como um tribunal técnico para o código. Trata-se da execução e do registo de testes de caixa-preta e integrados, mapeando evidências reais (como as URLs e endpoints do GitHub). O objetivo aqui é puramente analítico: validar se os requisitos de autenticação, controlo de versões, segurança e comunicação com a API passaram nos critérios de aceitação com o estatuto de [OK]. É a prova matemática e visual de que o sistema não possui erros críticos de integração.
- **6.2 Preparação e Entrada em Produção (A Esteira de Lançamento / Deployment):** Ao contrário dos testes da secção anterior, esta secção foca-se no **futuro imediato** da aplicação. Não se trata de descobrir se o código funciona (isso já foi provado na 6.1), mas sim de garantir o *Hardening* final e a estabilidade a longo prazo. Envolve rituais de infraestrutura como a auditoria de links quebrados, varrimento de segurança nas configurações finais, revisão minuciosa da documentação de entrega e congelamento do código (*code freeze*). É o passo burocrático e técnico que autoriza a viragem da chave para que o ecossistema passe de um projeto de laboratório para um produto disponível no mercado.

Matriz de Transição: Qualidade de Código vs. Operacionalização em Nuvem

A tabela abaixo sintetiza e confronta os objetivos e a natureza distinta de cada uma das etapas finais de entrega:

Critério Técnico	6.1 Checklist de Testes (Validação)	6.2 Preparação para Produção (Deployment)
Foco de Atuação	Verificação retroativa de erros, falhas e quebras de comunicação.	Certificação proativa de estabilidade, segurança e integridade de rede.
Natureza da Atividade	Empírica/Executiva: Rodar o sistema e testar fluxos reais (Login, API, CSS).	Governamental/Infras: Revisar permissões, documentação e blindar acessos.
Artefato de Saída	Relatório de evidências e tabela de conformidade funcional ([OK]).	Ambiente congelado, URL de produção ativa e homologação final de entrega.
Pergunta Respondida	"O sistema funciona como foi projetado e integra corretamente?"	"O ambiente está seguro e resiliente para o utilizador final utilizar em produção?"

7. Revisão Final e Apresentação

7.1 Vídeo de apresentação

<https://youtu.be/OFa3ZrZV1lg>

7.2 Roteiro da apresentação

1. Introdução e Contextualização
2. Planeamento, Cronograma e Arquitetura
3. Configuração Inicial e Rastreabilidade de Commits
4. Gestão de Segurança e o Papel do Gemini
5. Testes, Homologação e Demonstração
6. Conclusão e Encerramento

8. Conclusão

A implantação do ecossistema PortfolioHUB superou a mera entrega de uma interface estática, consolidando-se como um laboratório prático de aplicação de conceitos modernos de Engenharia de Software, Governação de Repositórios e Inteligência Artificial Aplicada. O sucesso do projeto materializa-se na convergência sinérgica entre o controle de versões, a automação de infraestrutura e a blindagem de segurança.

A análise crítica dos resultados obtidos permite extrair três pilares conclusivos:

- **1. Eficácia da Interoperabilidade e Automação (*Deploy*):** A integração nativa com o GitHub provou ser o cerne operacional da solução. A transição do fluxo local para o ambiente público em nuvem (via GitHub Pages) demonstrou como uma esteira de entrega contínua simplifica o gerenciamento de portfólios, garantindo que qualquer evolução de código seja refletida em tempo real e de forma transparente para o utilizador final.
- **2. Resiliência de Infraestrutura e Mitigação de Riscos:** O maior desafio técnico do projeto — que residia no correto balanceamento entre acessibilidade e proteção — foi solucionado através do *hardening* do repositório. A implementação do Princípio do Menor Privilégio nas permissões, aliada às barreiras ativas do 2FA e das *Branch Protection Rules*, transformou um ambiente potencialmente vulnerável num repositório auditável e seguro, alinhado com os padrões de conformidade do mercado tecnológico atual.
- **3. A IA Gemini como Catalisador de Produtividade:** A introdução do Google Gemini como agente consultor redefiniu a dinâmica de desenvolvimento acadêmico. A IA não operou como uma ferramenta de cópia automatizada, mas sim como um vetor de inteligência preventiva, otimizando o tempo de resposta na geração de scripts de configuração, sugerindo melhorias na arquitetura e elevando o rigor técnico da documentação final.

Em suma, o PortfolioHUB cumpre rigorosamente os objetivos propostos no seu planeamento. Mais do que uma plataforma concluída com estatuto de conformidade total [OK], o projeto deixa como principal legado a sedimentação de competências práticas indispensáveis para a atuação profissional na indústria de software: o desenvolvimento ágil, a cultura *DevOps* de versionamento frequente e a mentalidade de segurança por design (*Security by Design*).

9. Referências

- GitHub Documentation (Documentação do Trabalho): <https://docs.github.com>

- Google Gemini (Inteligência Artificial usada para o versionamento) : <https://gemini.google.com>
- Versões usadas do Gemini foi o 3.5 flash e 3.5 thinking.
- Documentação do PortfolioHUB usado para implementação: <https://github.com/vitoremanuel-dot/finala>
- GitHub: <https://github.com/vitoremanuel-dot>

10. Anexos (abaixo contém as fotos do desenvolvimento do projeto)

